Noise Suppression & Edge Detection

Image Processing Report

Name: Malak Mohamed Ahmed Mohamed | 20231540457

1. Overview

This report documents an image processing pipeline applied to two grayscale noisy images. For each processing stage, both a from-scratch implementation and the equivalent OpenCV built-in function are compared, demonstrating that both approaches produce identical results while building a solid understanding of the underlying algorithms.

Image	Description
Image 1	A statue portrait — moderate Gaussian noise, best suited for Gaussian filtering.
Image 2	A city skyline — heavier salt-and-pepper-style noise, best suited for Median filtering.

2. Loading the Images

The images are loaded using **cv2.imread** with the grayscale flag (0), producing 2D NumPy arrays of dtype uint8, which allows direct pixel-level operations.

```
noisy_image_pexels  = cv2.imread('noisy_image.png',  0)
noisy_image1_pexels = cv2.imread('noisy_image11.png', 0)
```

Reading as 2D arrays (height × width) simplifies all subsequent spatial filtering operations, since no channel dimension needs to be managed.

Image 1 — noisy_image.png (statue portrait, grayscale)

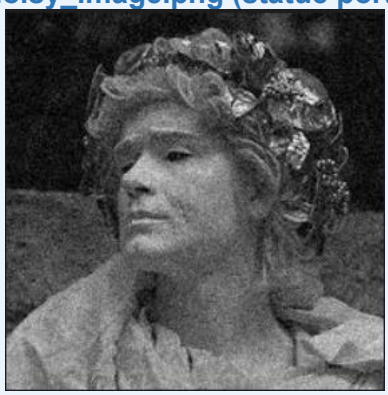


Image 2 — noisy_image11.png (city skyline, grayscale)

3. Noise Suppression

Three classical spatial filters were applied and compared: Box (Average) Filter, Gaussian Filter, and Median Filter. Each was implemented **from scratch** and then verified against the corresponding **OpenCV function**.

3.1 Box (Average) Filter

The box filter replaces each pixel with the arithmetic mean of all pixels within a rectangular neighborhood (kernel). Every neighbor is weighted equally, which is fast but tends to blur edges uniformly.

Algorithm (from scratch)

- Pad the image using reflect mode to handle borders without introducing artificial edges.
- Slide a kernel window of size k×k over every pixel.
- Compute np.mean() of the window and assign it to the output pixel.
- Clip results to [0, 255] and cast to uint8.

```
def box_filter(image, kernel_size):
    h, w = image.shape
    pad = kernel_size // 2
    padded = np.pad(image, ((pad,pad),(pad,pad)), mode='reflect')
    output = np.zeros_like(image, dtype=np.float32)
    for i in range(h):
        for j in range(w):
            output[i,j] = np.mean(padded[i:i+kernel_size, j:j+kernel_size])
    return np.clip(output, 0, 255).astype(np.uint8)
```

OpenCV equivalent

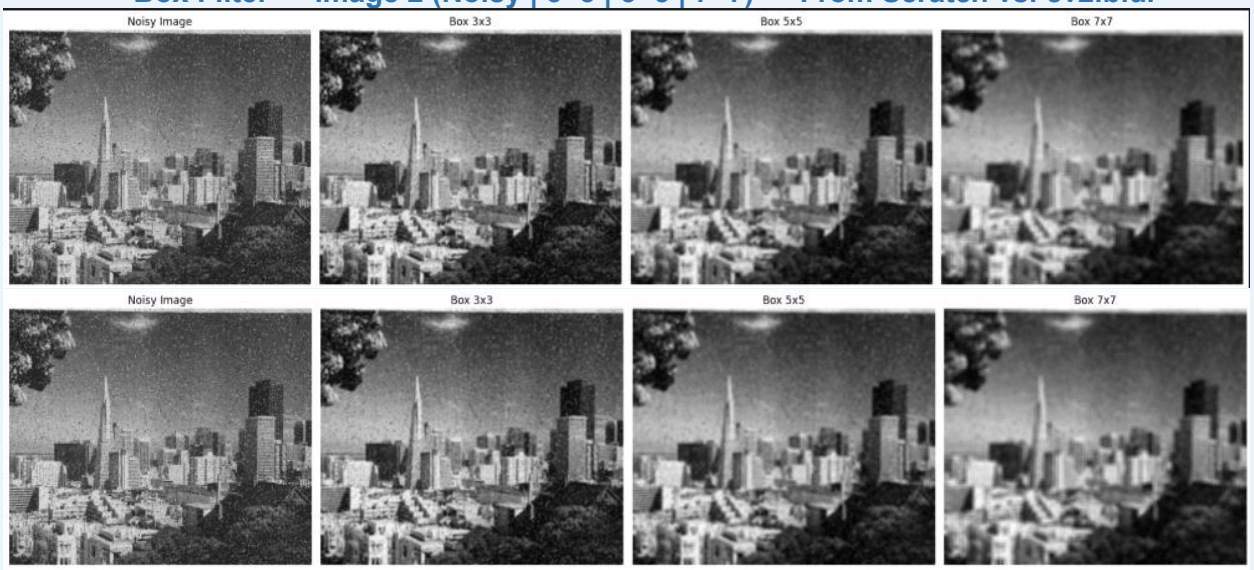
```
cv2.blur(image, (kernel_size, kernel_size))
```

Results — Image 1

Box Filter — Image 1 (Noisy | 3×3 | 5×5 | 7×7) — From Scratch vs. cv2.blur

Results — Image 2

Box Filter — Image 2 (Noisy | 3×3 | 5×5 | 7×7) — From Scratch vs. cv2.blur



Comparison

Both implementations produced identical outputs across all kernel sizes (3×3, 5×5, 7×7). While the box filter successfully reduces noise, it applies equal weight to all neighbors, making it prone to over-smoothing and loss of edge sharpness — especially at larger kernel sizes. It is generally not the best choice for these images.

3.2 Gaussian Filter

The Gaussian filter applies a weighted average where pixels closer to the center of the kernel contribute more than those farther away, following the 2D Gaussian (bell-curve) distribution. This produces more natural-looking smoothing than the box filter.

Gaussian formula used

$$G(x, y) = \exp(-(x^2 + y^2) / (2\sigma^2))$$

The constant $1/(2\pi\sigma^2)$ is omitted because the kernel is normalized by dividing all values by their sum, cancelling the constant.

Kernel size rule

Kernel size is derived from sigma using the 6σ rule (kernel_size = int(6 * sigma), rounded up to the nearest odd number). This captures 99.7% of the Gaussian distribution (±3σ on each side), making values outside the kernel negligible.

σ (Sigma)	Kernel Size
σ = 1	7 × 7
σ = 2	13 × 13
σ = 3	19 × 19

Algorithm (from scratch)

- Build the kernel by evaluating the Gaussian formula at each offset (x, y) from the center.
- Normalize the kernel so all values sum to 1.
- Pad the image and slide the kernel, computing np.sum(window * kernel) at each position.

```
def gaussian_kernel(sigma):
    kernel_size = int(6 * sigma)
    if kernel_size % 2 == 0: kernel_size += 1
    pad = kernel_size // 2
    kernel = np.zeros((kernel_size, kernel_size), dtype=np.float32)
    for x in range(-pad, pad+1):
        for y in range(-pad, pad+1):
            kernel[x+pad,y+pad] = np.exp(-(x**2+y**2)/(2*sigma**2))
    return kernel / kernel.sum(), kernel_size
```

OpenCV equivalent

```
cv2.GaussianBlur(image, (kernel_size, kernel_size), sigma)
```

Results — Image 1

Gaussian Filter — Image 1 (Noisy | σ=1 | σ=2 | σ=3) — From Scratch

Gaussian Filter — Image 1 (Noisy | σ=1 | σ=2 | σ=3) — cv2.GaussianBlur

Results — Image 2

Gaussian Filter — Image 2 (Noisy | σ=1 | σ=2 | σ=3) — From Scratch

Gaussian Filter — Image 2 (Noisy | σ=1 | σ=2 | σ=3) — cv2.GaussianBlur

Comparison

Both approaches gave identical results. The Gaussian filter outperformed the box filter by preserving more edge detail while still reducing noise effectively. It was selected as the best filter for Image 1 (the statue portrait).

3.3 Median Filter

The median filter replaces each pixel with the median value of its neighborhood rather than the mean. Because the median is robust to outliers, this filter is particularly effective at removing salt-and-pepper noise without significantly blurring edges.

Algorithm (from scratch)

- Pad the image using reflect mode.
- Slide a k×k window over every pixel.
- Sort all values in the window and pick the middle value using np.median().
- Clip and cast to uint8.

```
def median_filter(image, kernel_size):
    h, w = image.shape
    pad = kernel_size // 2
    padded = np.pad(image, ((pad,pad),(pad,pad)), mode='reflect')
    output = np.zeros_like(image, dtype=np.float32)
    for i in range(h):
        for j in range(w):
            output[i,j] = np.median(padded[i:i+kernel_size, j:j+kernel_size])
    return np.clip(output, 0, 255).astype(np.uint8)
```

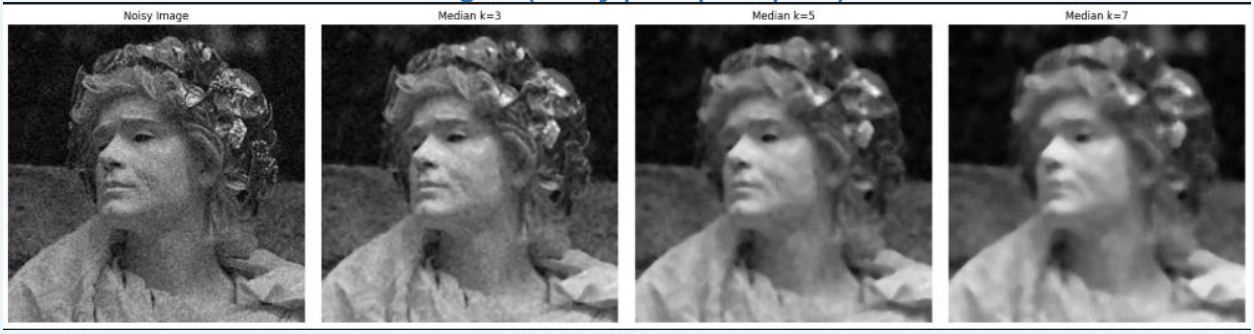
OpenCV equivalent

```
cv2.medianBlur(image, ksize=kernel_size)
```

Results — Image 1

Median Filter — Image 1 (Noisy | 3×3 | 5×5 | 7×7) — From Scratch

Median Filter — Image 1 (Noisy | 3×3 | 5×5 | 7×7) — cv2.medianBlur



Results — Image 2

Median Filter — Image 2 (Noisy | 3×3 | 5×5 | 7×7) — From Scratch

Median Filter — Image 2 (Noisy | 3×3 | 5×5 | 7×7) — cv2.medianBlur

Comparison

Both approaches yielded identical results. The median filter gave the best output for Image 2 (the city skyline), preserving sharp building edges while effectively suppressing noise.

3.4 Choosing the Best Filter

After comparing all three filters across both images, the following selections were made and carried forward to all subsequent processing stages:

Image	Best Filter	Reason
Image 1 — Statue portrait	Gaussian Filter (σ = 1)	Best noise reduction while preserving edge structure
Image 2 — City skyline	Median Filter (3×3)	Effective against impulse noise, sharp edges preserved

Best Smoothed Results — Image 1 (Gaussian σ=1) and Image 2 (Median 3×3)

4. Computing Gradient Magnitude & Direction

The Sobel operator is applied to the smoothed images to detect edges by computing the intensity gradient in the horizontal (x) and vertical (y) directions.

Sobel kernels

```
kernel_x = [[-1, 0, 1], [-2, 0, 2], [-1, 0, 1]]   → horizontal gradient
kernel_y = [[-1,-2,-1], [ 0, 0, 0], [ 1, 2, 1]]   → vertical gradient
```

Algorithm (from scratch)

- Convolve the smoothed image with each Sobel kernel using a sliding window (convolve2d).
- Compute the gradient magnitude: $M = \sqrt{(Gx^2 + Gy^2)}$.
- Compute the gradient direction: $\theta = \arctan2(Gy, Gx)$ converted to degrees and mapped to [0°, 360°].

```
image1_magnitude = np.sqrt(grad_x**2 + grad_y**2)
image1_direction = np.arctan2(grad_y, grad_x) * (180 / np.pi) % 360
```

OpenCV equivalent

```
sobel_x   = cv2.Sobel(image, cv2.CV_64F, 1, 0, ksize=3)
sobel_y   = cv2.Sobel(image, cv2.CV_64F, 0, 1, ksize=3)
magnitude = cv2.magnitude(sobel_x, sobel_y)
direction = cv2.phase(sobel_x, sobel_y, angleInDegrees=True)
```

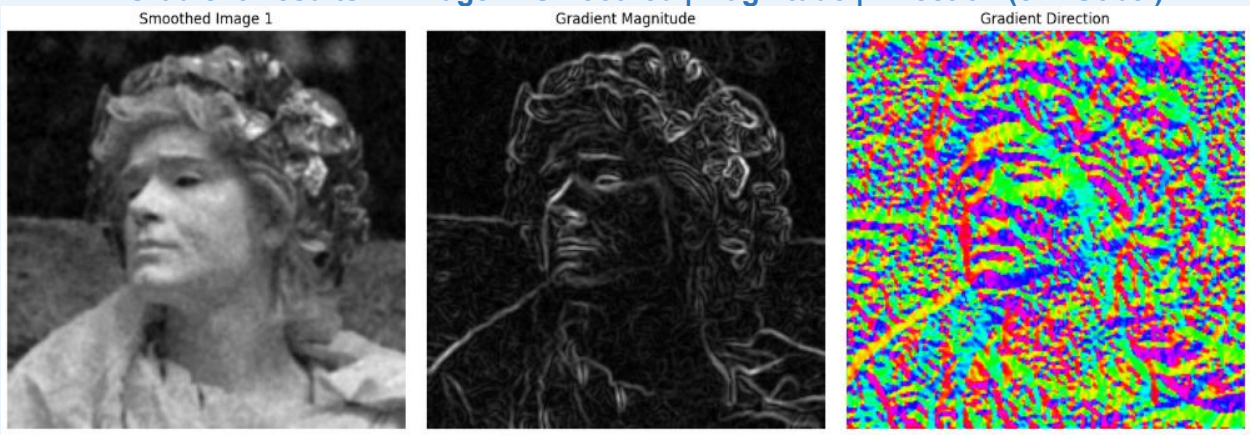
Results

Image	Magnitude Range	Direction Range
Image 1	0.0 → 452.04	0° → 359.6°
Image 2	0.0 → 633.66	0° → 359.9°

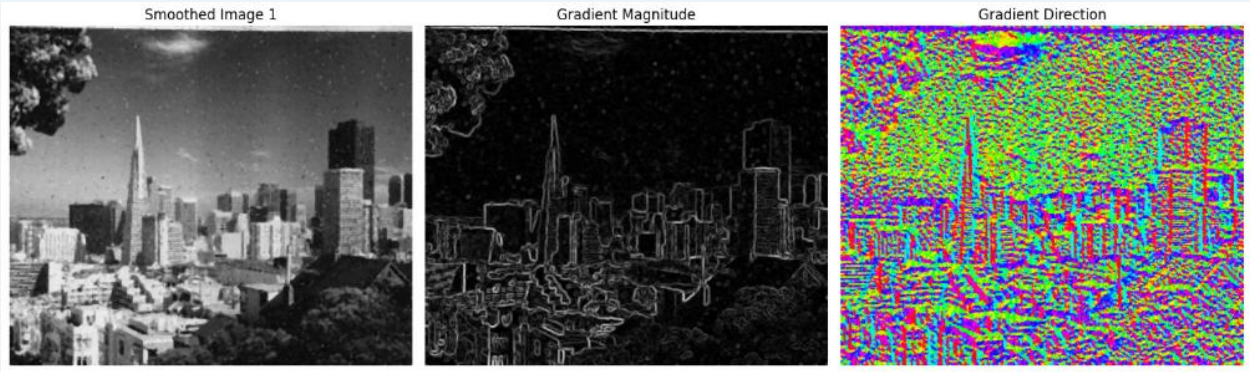
Both scratch and OpenCV implementations produced identical magnitude and direction values. Direction is visualized using the HSV colormap, where each hue represents a different edge angle.

Gradient Results — Image 1: Smoothed | Magnitude | Direction (From Scratch)

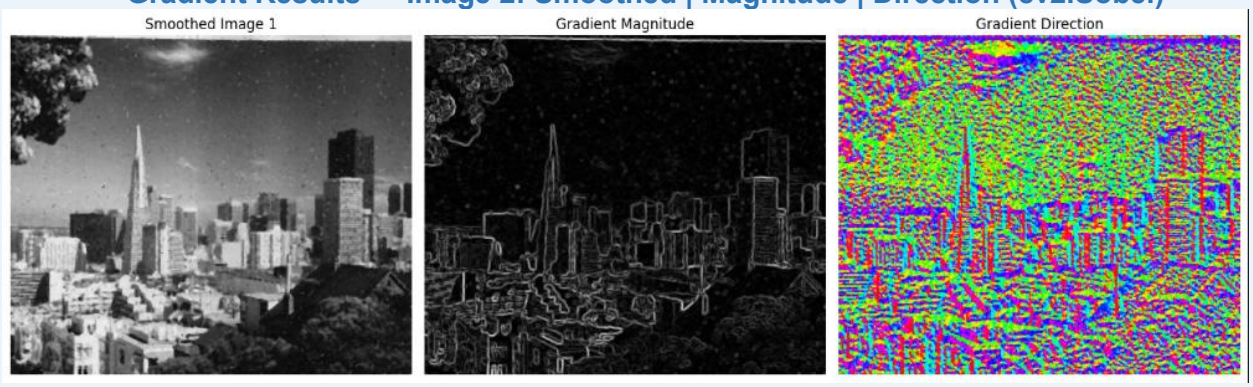
Gradient Results — Image 1: Smoothed | Magnitude | Direction (cv2.Sobel)



Gradient Results — Image 2: Smoothed | Magnitude | Direction (From Scratch)



Gradient Results — Image 2: Smoothed | Magnitude | Direction (cv2.Sobel)



5. Non-Maximum Suppression (NMS)

NMS thins thick edges by keeping only the local maximum pixel along the gradient direction and suppressing all others. This eliminates double edges and produces single-pixel-wide edge contours.

Algorithm (from scratch)

- Normalize the direction to [0°, 180°) since edges are direction-symmetric.
- Quantize each angle into one of four directions: 0°, 45°, 90°, or 135°.
- For each pixel, compare its magnitude to the two neighbors along its gradient direction.
- Keep the pixel if it is the local maximum; suppress it (set to 0) otherwise.

Angle Range	Neighbors Compared
0° / 180° (horizontal edge)	Left & right
45° (diagonal /)	Top-right & bottom-left
90° (vertical edge)	Top & bottom
135° (diagonal \)	Top-left & bottom-right

```
nms_image1 = non_maximum_suppression(image1_magnitude, image1_direction)
nms_image2 = non_maximum_suppression(image2_magnitude, image2_direction)
```

NMS — Image 1: Smoothed | Magnitude (before NMS) | After NMS (thinned edges)

NMS — Image 2: Smoothed | Magnitude (before NMS) | After NMS (thinned edges)

6. Hysteresis Thresholding

Hysteresis thresholding classifies edge pixels using two thresholds: a high threshold for strong (certain) edges and a low threshold for weak (candidate) edges. Weak edges are only kept if they are connected to a strong edge, effectively removing isolated noise while preserving continuous edges.

Algorithm (from scratch)

- Label pixels with magnitude ≥ high_thresh as strong (255).
- Label pixels with low_thresh ≤ magnitude < high_thresh as weak (75).
- For each weak pixel, check its 3×3 neighborhood — promote to strong if any neighbor is strong, suppress otherwise.

```
def hysteresis_thresholding(nms, low_thresh, high_thresh):
    strong_mask = nms >= high_thresh
    weak_mask   = (nms >= low_thresh) & (nms < high_thresh)
    output[strong_mask] = 255
    output[weak_mask]   = 75
    # promote weak pixels connected to strong ones
```

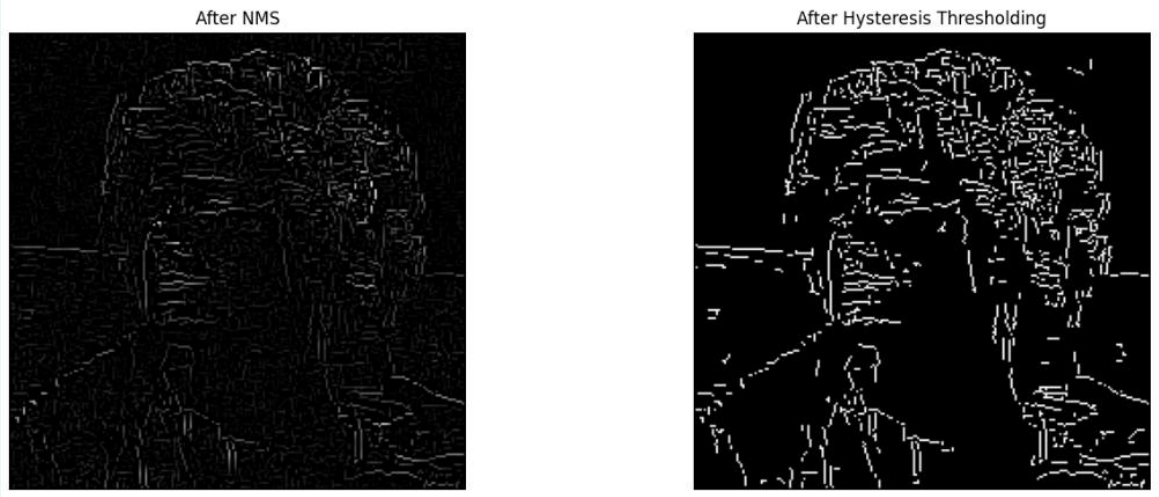
Threshold tuning — Image 1

low / high	Observation
30 / 100	Too many noise edges retained
40 / 150	Clean but some real edges lost
40 / 87	Best balance — selected

Hysteresis — Image 1: low=30/high=100

Hysteresis — Image 1: low=40/high=150

Hysteresis — Image 1: low=40/high=87 (best)



Threshold tuning — Image 2

low / high	Observation
30 / 100	Excessive noise edges
50 / 150	Good clarity — selected
70 / 180	Too aggressive, edges broken

Hysteresis — Image 2: low=30/high=100

Hysteresis — Image 2: low=70/high=180

Hysteresis — Image 2: low=50/high=150 (best)

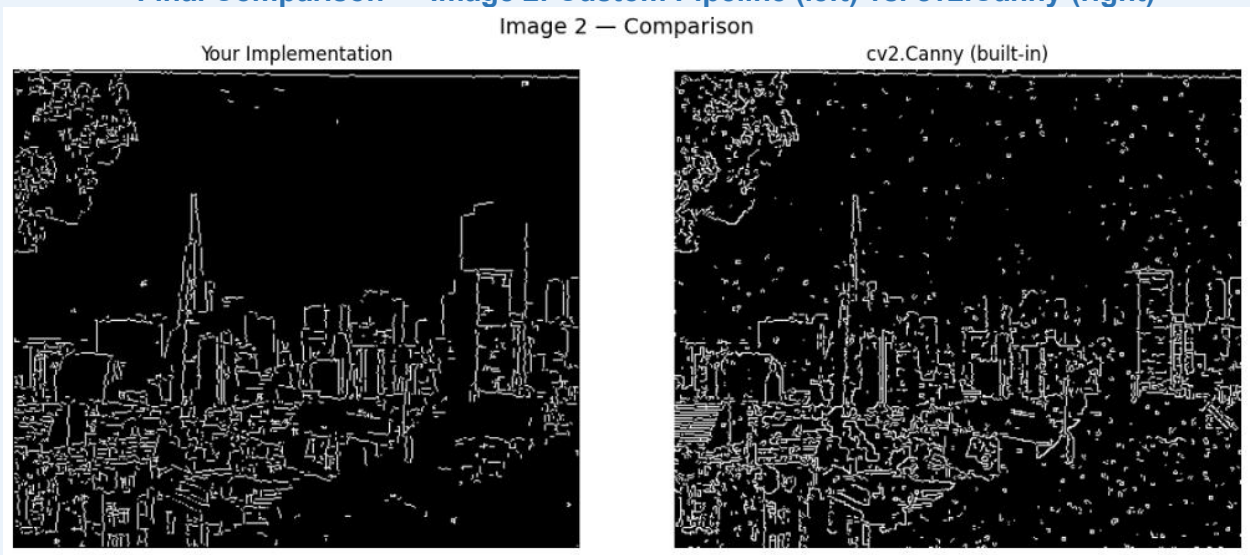
7. Final Comparison: Custom Implementation vs. cv2.Canny

The full pipeline (Gaussian/Median smoothing → Sobel gradients → NMS → Hysteresis thresholding) is compared side-by-side with OpenCV's cv2.Canny built-in function.

```
canny_image1 = cv2.Canny(noisy_image_pexels,  threshold1=170, threshold2=260)
canny_image2 = cv2.Canny(noisy_image1_pexels, threshold1=250, threshold2=350)
```

Final Comparison — Image 1: Custom Pipeline (left) vs. cv2.Canny (right)

Final Comparison — Image 2: Custom Pipeline (left) vs. cv2.Canny (right)



Both implementations detect the same structural edges in each image. The custom pipeline required manual tuning of each stage, while cv2.Canny encapsulates all steps internally. The close visual similarity between the two outputs validates the correctness of the from-scratch implementation.

Aspect	Custom Pipeline vs. cv2.Canny
Edge locations	Visually equivalent — same structure detected
Control	Custom: full per-stage control; Canny: single function
Tuning	Custom: threshold per stage; Canny: two thresholds only
Speed	Custom (pure Python): slower; Canny (C++): much faster

8. Conclusion

This project demonstrated a complete image processing pipeline from raw noisy images to clean edge maps, implemented both from scratch and using OpenCV, confirming that both approaches produce equivalent results.

- Box filter: simple but over-smooths; not optimal for either image.
- Gaussian filter: best for Image 1 — reduces Gaussian noise while preserving structure.
- Median filter: best for Image 2 — effective against impulse/salt-and-pepper noise without blurring edges.
- Sobel gradients, NMS, and hysteresis together form the classical Canny edge detection pipeline.
- The custom implementation closely matches cv2.Canny, validating all from-scratch stages.